

SKEPTICAL

Introduction

Brief description of the design philosophy of your language.

Skeptical is a structured, English-like programming language designed to be highly readable and verbose like COBOL, while incorporating the scoping mechanism Python. It uses indentation to define blocks, supports lexical scoping, and encourages a top-down, procedural style of development

Discussion of the languages that influenced your language design.

The purpose of Skeptical is to offer the readability and documentation-style syntax of COBOL, with Python's modern ease of use through dynamic typing, lexical scoping behavior and comparison operators. We also implement const variables and type "locking" to allow for better memory management and easier debugging.

Design Influences

Cobol-Structure, division-based organization, focuses on forms.

Python- Scoping, simplicity, indentation-based syntax, lack of required type declarations.

EBNF Description of your language syntax.

```
program          = { division } ;

division         = static_division | dynamic_division;

static_division  = "STATIC DIVISION." newline
                  { id_statement } ;

Stadecl         = "PROGRAM-ID." identifier "." newline
                  | "AUTHOR." string "." newline
                  | "DATE-WRITTEN." string "." newline
                  | constant;

constant        = "FIX" identifier "TO" expression "." ;

dynamic_division = "DYNAMIC DIVISION." newline
                  { statement } ;

statement       = assignment
                  | print_stmt
                  | input_stmt
                  | if_stmt
```

| loop_stmt
| call_stmt
| function_def
| random_stmt;

assignment = "SET" identifier "TO" expression ["AS" term] "." ;

print_stmt = "DISPLAY" output "." ;

input_stmt = "PROMPT" identifier "WITH" string "." ;

if_stmt = "IF" expression "THEN" newline
indent { statement } dedent
["ELSE" newline indent { statement } dedent] ;

loop_stmt = "START" identifier "AS" number "TO" number "DOING" newline
indent { statement } dedent ;

call_stmt = "CALL" identifier ["WITH" arglist] "." newline ;

function_def = "FUNCTION" identifier ["WITH" arglist] "." newline
indent { statement } dedent
"RETURN" identifier "." ;

random_stmt = "SET" identifier "TO RANDOM" number "TO" number "." ;

arglist = expression { "," expression } ;

output =string | identifier;

expression = disjunction ;

disjunction = conjunction { "OR" conjunction } ;

conjunction = comparison { "AND" comparison } ;

comparison = sum { ("==" | "!=" | "<" | "<=" | ">" | ">=") sum } ;

sum = term { ("+" | "-") term } ;

term = power { ("*" | "/" | "MOD") power } ;

power = factor ["****" power] ;

```

factor      = number
             | identifier
             | "(" expression ")"
             | "-" factor ;

identifier   = letter { letter | digit | "_" } ;
string       = "\"" { character } "\"" ;
number       = [ "-" ] digit { digit } [ "." digit { digit } ] ;

character    = letter | digit | symbol ;

symbol       = "!" | "@" | "#" | "$" | "%" | "^" | "&" | "*" | "(" | ")" | "-"
             | "_" | "+" | "=" | "{" | "}" | "[" | "]" | "|" | "\" | ":"
             | ";" | "'" | "," | "." | "/" | "<" | ">" | "?" | " " ;

letter       = "a".."z" | "A".."Z" ;
digit        = "0".."9" ;
nonzero digit = "1"..."9"
newline      = "\n" ;
indent       = "\t" ;
dedent       = "" ;

comment = "#" { character } newline ;

```

Semantic Behavior Description

Describing Variables

Variables are implicitly declared at first use (like Python). They are lexically scoped—that is, a variable is accessible within the block in which it is defined and in any nested (child) blocks.

Example:

```

SET x TO 5.
IF x > 3 THEN
    SET y TO x + 1.

```

DISPLAY y.

Describe Your Control Structures

Functions

Functions are defined with the FUNCTION keyword. They group statements and may accept parameters. The syntax is:

```
FUNCTION function_name [WITH arg1, arg2, ...].  
    <statements>
```

```
RETURN result.
```

Example:

```
FUNCTION double WITH n,m.  
    SET result TO n * m.  
RETURN result.
```

If Statements

Conditional execution uses IF, THEN, and an optional ELSE:

```
IF condition THEN  
    <statements>  
ELSE  
    <statements>
```

Example:

```
IF x > 10 THEN  
    DISPLAY "x is greater than 10."  
ELSE  
    DISPLAY "x is 10 or less."
```

Loops

Loops iterate over a sequence of numbers with a syntax modeled after a "for" loop:

```
FROM identifier EQUALS expression TO expression DO  
    <statements>
```

Example:

```
FROM i EQUALS 1 TO 10 DO  
    DISPLAY i.
```

Expression Evaluation and Operators

Arithmetic Operators: +, -, *, /, MOD perform standard arithmetic.

Comparison Operators: ==, !=, <, <=, >, >= compare values.

Logical Operators: AND, OR combine conditions.

Define What Every Keyword Means

STATIC DIVISION. Contains static declarations (e.g., program name, author, date written, constants).

PROGRAM-ID. Specifies the program's name.

AUTHOR. Declares the program's author.

DATE-WRITTEN. Specifies the program's creation date.

DYNAMIC DIVISION. Contains the executable code (the program's instructions).

FIX Declares a constant (a value that won't change) using the syntax: FIX var TO expression.

SET Assigns a value to a variable (e.g., SET x TO 5.).
TO,AS Used in assignments and loops (e.g., SET x TO 10, FROM i AS 1 TO 10 DOING).
WITH Used in input and optionally function definition to pass parameters
DISPLAY Outputs the value of an expression to the screen.
PROMPT Prompts the user for input (e.g., PROMPT name WITH "Enter your name:").
IF, THEN, ELSE Executes conditional code: if a condition is true, the code after THEN is executed; otherwise, code after ELSE is executed (if present).
START, DOING Controls loop execution, iterating from one value to another.
CALL Invokes a function, optionally passing arguments.
FUNCTION Defines a function with a name and a block of statements.
RETURN Terminates a function and returns a value.
RANDOM Generates a random number between two given values (inclusive).
MOD Calculates the remainder of a division (e.g., 5 MOD 3 equals 2).
Begins a comment; everything on that line after # is ignored.

Code Examples

Hello, World

STATIC DIVISION.
PROGRAM-ID. HelloWorld.
DATE-WRITTEN. 2025-04-06.

DYNAMIC DIVISION.
DISPLAY "Hello, World!".

Roll Routine (Generate a Random Number from 1–6)

STATIC DIVISION.
PROGRAM-ID. RollDie.
DATE-WRITTEN. 2025-04-06.

DYNAMIC DIVISION.
SET roll_result TO RANDOM 1 TO 6.
DISPLAY "You rolled: " roll_result.

Count from a Starting Number to an Ending Number and Check Even/Odd

STATIC DIVISION.
PROGRAM-ID. CountEvenOdd.
DATE-WRITTEN. 2025-04-06.

DYNAMIC DIVISION.
PROMPT start_num WITH "Enter the starting number:".
PROMPT end_num WITH "Enter the ending number:".

START i AS start_num TO end_num DOING
IF i MOD 2 == 0 THEN
DISPLAY i " is even.".

ELSE

DISPLAY i " is odd."

Full-Feature Example (Functions, Loops, and Conditionals)

STATIC DIVISION.

PROGRAM-ID. FeatureDemo.

AUTHOR. "Test Author".

DATE-WRITTEN. "2025-04-10".

FIX LIMIT TO 5.

DYNAMIC DIVISION.

Define a function to double a number.

FUNCTION double WITH n.

SET result TO n * 2.

RETURN result.

PROMPT usernum WITH "Enter a number:".

SET doubled TO CALL double WITH usernum.

DISPLAY "Double is: " doubled.

Generate and display a random value.

SET rand_val TO RANDOM 100 TO 200.

DISPLAY "Random value: " rand_val.

Loop from 1 to the fixed limit, demonstrating conditional logic.

START i AS 1 TO LIMIT DOING

IF i != 3 THEN

DISPLAY i.

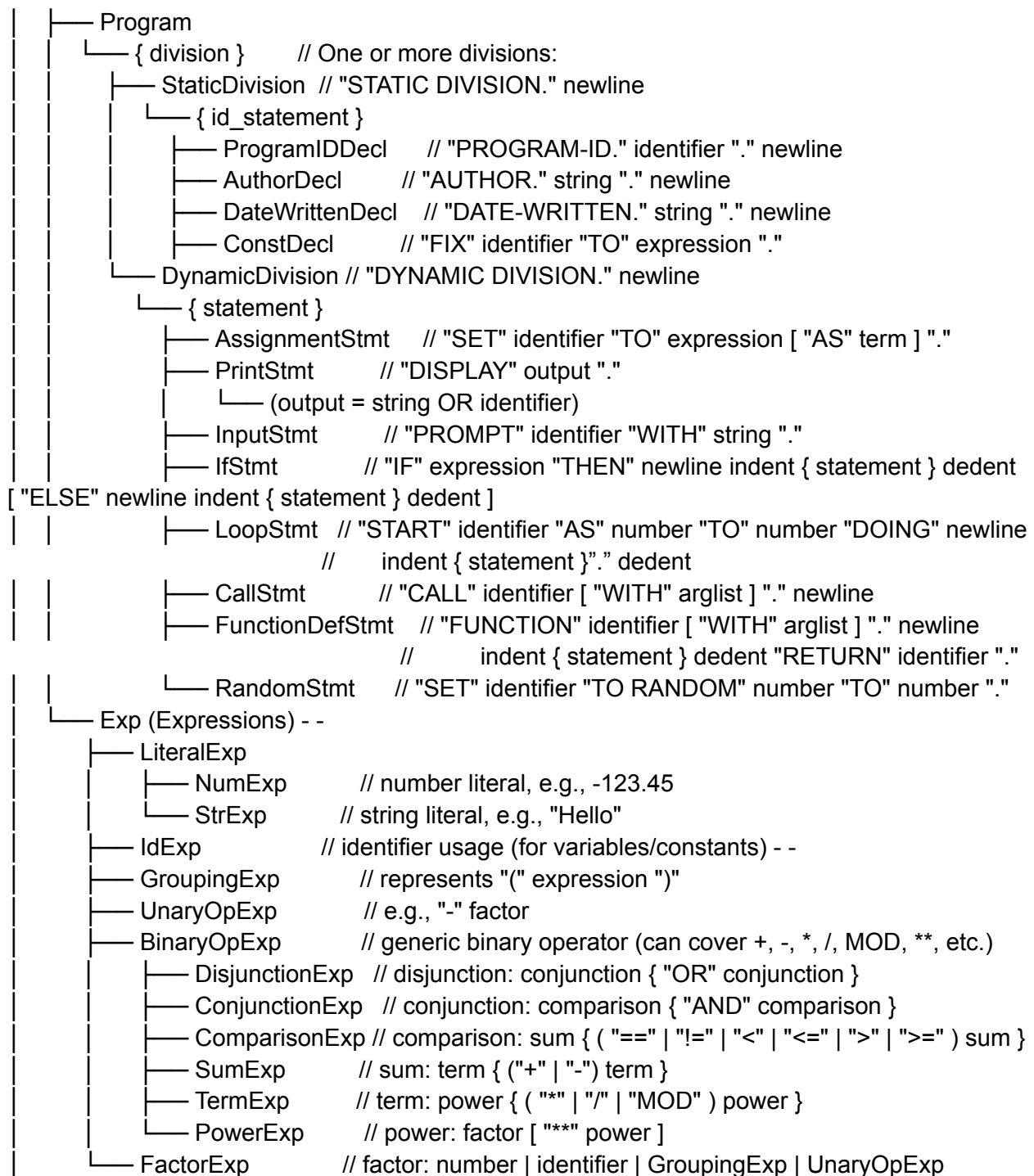
ELSE

DISPLAY "Three!".

Things to figure out- what do i need to do with exp, am i doing the AST right(review it), review my evaluator to ensure i am doing that right, what do i need to do with the printer?

AST

|— ASTNode



```

package skeptical;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class Evaluator implements AST.Visitor<Object> {

    private Env env;

    Public class Evaluator(Env env) {
        this.env = env;
    }

    // Method to evaluate a Program
    @Override
    public Object visit(AST.Program p, Env env) {
        // Visit static and dynamic divisions
        p.staticDivision().accept(this, env);
        p.dynamicDivision().accept(this, env);
        return null;
    }

    // Static Division: Evaluate the static declarations (like constants)
    @Override
    public Object visit(AST.StaDiv d, Env env) {
        for (AST.StaDecl decl : d.getStatements()) {
            decl.accept(this, env); // Evaluate each static declaration
        }
        return null;
    }

    // Dynamic Division: Evaluate the dynamic statements (like assignments)
    @Override
    public Object visit(AST.DynDiv d, Env env) {
        for (AST.Statement stmt : d.getStatements()) {
            stmt.accept(this, env); // Evaluate each dynamic statement
        }
        return null;
    }

    // Handling Static Declarations

```



```

@Override
public Object visit(AST.ProgId sd, Env env) {
    // Handle program identifier (name declaration)
    return null;
}

@Override
public Object visit(AST.Auth sd, Env env) {
    // Handle authorization (perhaps logging or storing user info)
    return null;
}

@Override
public Object visit(AST.Date sd, Env env) {
    // Handle date (maybe store or validate)
    return null;
}

@Override
public Object visit(AST.Const sd, Env env) {
    // Evaluate the constant declaration
    double value = (double) sd.value().accept(this, env);
    env.put(sd.id(), value); // Store the constant value in the environment
    return null;
}

// Expressions
@Override
public Object visit(AST.IdExp e, Env env) {
    return env.get(e.id()); // Retrieve the value from the environment
}

@Override
public Object visit(AST.NumExp e, Env env) {
    return e.value(); // Return the number directly
}

@Override
public Object visit(AST.StrExp e, Env env) {
    return e.value(); // Return the string directly
}

// Binary Expressions
@Override

```

```

public Object visit(AST.DisjExp e, Env env) {
    boolean left = (boolean) e.left().accept(this, env);
    boolean right = (boolean) e.right().accept(this, env);
    return left || right; // Logical OR
}

```

```

@Override
public Object visit(AST.ConjExp e, Env env) {
    boolean left = (boolean) e.left().accept(this, env);
    boolean right = (boolean) e.right().accept(this, env);
    return left && right; // Logical AND
}

```

```

@Override
public Object visit(AST.CompExp e, Env env) {
    double left = (double) e.left().accept(this, env);
    double right = (double) e.right().accept(this, env);
    switch (e.operator()) {
        case "==":
            return left == right;
        case "!=":
            return left != right;
        case "<":
            return left < right;
        case ">":
            return left > right;
        case "<=":
            return left <= right;
        case ">=":
            return left >= right;
        default:
            throw new UnsupportedOperationException("Unknown operator: " + e.operator());
    }
}

```

```

@Override
public Object visit(AST.SumExp e, Env env) {
    double left = (double) e.left().accept(this, env);
    double right = (double) e.right().accept(this, env);
    if (e.operator().equals("+")) {
        return left + right; // Addition
    } else if (e.operator().equals("-")) {
        return left - right; // Subtraction
    } else {

```

```

        throw new UnsupportedOperationException("Unknown operator: " + e.operator());
    }
}

```

```

@Override
public Object visit(AST.TermExp e, Env env) {
    double left = (double) e.left().accept(this, env);
    double right = (double) e.right().accept(this, env);
    if (e.operator().equals("*")) {
        return left * right; // Multiplication
    } else if (e.operator().equals("/")) {
        return left / right; // Division
    } else {
        throw new UnsupportedOperationException("Unknown operator: " + e.operator());
    }
}

```

```

@Override
public Object visit(AST.PowExp e, Env env) {
    double left = (double) e.left().accept(this, env);
    double right = (double) e.right().accept(this, env);
    return Math.pow(left, right); // Power operation
}

```

// Unary Expression

```

@Override
public Object visit(AST.UnaryOpExp e, Env env) {
    double value = (double) e.getExpr().accept(this, env);
    if (e.getOperator().equals("-")) {
        return -value; // Unary negation
    } else {
        throw new UnsupportedOperationException("Unknown operator: " + e.getOperator());
    }
}

```

// Factor Expressions (e.g., values, grouping)

```

@Override
public Object visit(AST.FactorExp e, Env env) {
    return e.getExpr().accept(this, env); // Evaluate the contained expression
}

```

```

@Override
public Object visit(AST.GroupExp e, Env env) {
    return e.getExpr().accept(this, env); // Evaluate the grouped expression
}

```

```
}
```

```
// Evaluating Statements
```

```
@Override
```

```
public Object visit(AST.Assign stmt, Env env) {  
    double value = (double) stmt.getExpression().accept(this, env);  
    env.put(stmt.getIdentifier(), value); // Assign the value to the identifier  
    return null;  
}
```

```
@Override
```

```
public Object visit(AST.Print stmt, Env env) {  
    Object value = stmt.getOutput().accept(this, env);  
    System.out.println(value); // Print the output  
    return null;  
}
```

```
@Override
```

```
public Object visit(AST.Input stmt, Env env) {  
    // Handle input by prompting user (or simulation of input)  
    return null;  
}
```

```
@Override
```

```
public Object visit(AST.IfStmt stmt, Env env) {  
    boolean condition = (boolean) stmt.getCondition().accept(this, env);  
    if (condition) {  
        for (AST.Statement s : stmt.getThenBranch()) {  
            s.accept(this, env); // Execute the then branch  
        }  
    } else if (stmt.getElseBranch() != null) {  
        for (AST.Statement s : stmt.getElseBranch()) {  
            s.accept(this, env); // Execute the else branch  
        }  
    }  
    return null;  
}
```

```
@Override
```

```
public Object visit(AST.LoopStmt stmt, Env env) {  
    for (int i = stmt.getStartValue(); i <= stmt.getEndValue(); i++) {  
        for (AST.Statement s : stmt.getBody()) {  
            s.accept(this, env); // Execute each statement in the loop  
        }  
    }  
}
```

```

    }
    return null;
}

@Override
public Object visit(AST.CallStmt stmt, Env env) {
    // Handle function call
    return null;
}

@Override
public Object visit(AST.FuncDef stmt, Env env) {
    // Define function (can be stored in environment or used later)
    return null;
}

@Override
public Object visit(AST.Rand stmt, Env env) {
    int randomValue = (int) (Math.random() * (stmt.getMax() - stmt.getMin() + 1)) +
stmt.getMin();
    env.put(stmt.getIdentifier(), randomValue); // Assign random value
    return null;
}
}

// The Env class: Environment to store variable values
class Env {
    private final Map<String, Object> variables = new HashMap<>();

    public void put(String key, Object value) {
        variables.put(key, value);
    }

    public Object get(String key) {
        return variables.get(key);
    }
}

```

```
class Rand extends Statement {
    public String identifier;
    public int min;
    public int max;

    // Constructor
    public Rand(String identifier, int min, int max) {
        this.identifier = identifier;
        this.min = min;
        this.max = max;
    }

    // Method to generate a random number
    public int generateRandom() {
        Random rand = new Random();
        return rand.nextInt((max - min) + 1) + min; // Returns a value between min and max,
inclusive
    }
}
```